

Herald



Protocol Research: Server-Sent Events (SSE)

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	SSE (WHATWG HTML spec / W3C "EventSource") is a one-way streaming protocol over plain HTTP. Browsers natively support it via new <code>EventSource(url)</code> . For Herald, SSE is the simplest way to expose READ-ONLY live streams (dashboards, status pulls) without the WS handshake complexity. SSE uses standard HTTP/2 — no upgrade dance, friendly to load balancers + reverse proxies. Wave 4e (alongside WS).
Issues	open-questions: heartbeat cadence; max connection lifetime; Last-Event-ID resumption.
Continuation	Wave 4e — open HRD-303..HRD-310; new module <code>commons_sse/</code> .

Constitutional anchors

- **§107 anti-bluff** — SSE tests use real `curl -N` + a real browser EventSource page; assert `text/event-stream` content-type + correct framing.
- **§11.4.74 catalogue-check** — no SSE module needed; Go `stdlib net/http + http.Flusher` suffices.
- **§11.4.61** — tracked doc.

Table of contents

- [§1. Protocol overview](#)
- [§2. Specification deep-dive](#)
- [§3. Herald-specific applicability analysis](#)
- [§4. Step-by-step implementation guide for Herald](#)
- [§5. §107 anti-bluff testing strategy](#)
- [§6. Open questions for operator](#)
- [§7. References](#)
- [§8. Catalogue-check verdict](#)

§1. Protocol overview

Server-Sent Events (SSE), also known as the **EventSource API**, is a server-push streaming protocol defined in the WHATWG HTML Living Standard (§9.2 "Server-sent events"). Born around 2009-2011, ubiquitously supported in browsers since IE10. License: open standard.

Wire format. Plain text over HTTP. Content-Type: `text/event-stream`. Events are blank-line-separated; each event has zero or more fields:

```
data: {"foo": 1}
id: 42
event: notification
retry: 5000
```

```
data: {"foo": 2}
```

Direction. ONE-WAY (server → client). For client → server, use a separate POST. (Compare WS: bidirectional.)

Transport. Plain HTTP/1.1 or HTTP/2. No upgrade handshake. Just a regular GET with `Accept: text/event-stream`, server keeps the connection open + streams.

Reconnection. Built into EventSource: on disconnect, browser auto-reconnects after `retry: N` ms and sends `Last-Event-ID: <id>` header to let the server resume.

Adoption. Browsers (native). MCP (legacy SSE transport before Streamable HTTP). OpenAI's streaming completions API. CloudFront live events. Pretty much every "live updates" dashboard.

Pros over WS. Simpler. No upgrade. Native browser reconnection. Works through proxies/load balancers without WS support. HTTP/2 multiplexes 100 SSE streams over one connection.

Cons. One-way. Text only (no binary, though base64 works). Browser limit of 6 concurrent EventSources per origin over HTTP/1.1 (HTTP/2 fixes this).

§2. Specification deep-dive

§2.1 Event format

```
field: value\n
field: value\n
\n    <- blank line marks end of event
```

Fields:

- `data` — payload (multiline allowed; each `data:` line concatenated with newlines).
- `event` — event name (default `message`).
- `id` — last event ID (browser sends in `Last-Event-ID` on reconnect).
- `retry` — reconnection delay in ms.

Comments: lines starting with `:` (e.g. `: heartbeat`).

§2.2 Response headers

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

§2.3 Client (browser) usage

```
const es = new EventSource('/events?token=<jwt>', {
  withCredentials: true });
es.onmessage = e => console.log(JSON.parse(e.data));
es.addEventListener('notification', e => ...);
```

§2.4 Reconnection + resumption

If the connection drops:

1. Browser waits `retry` ms (default 3000).
2. Browser reconnects.
3. If browser saw any `id:` field, it sends `Last-Event-ID: <last-id>`.
4. Server SHOULD resume from after that ID.

Critical for "always-on" dashboards.

§2.5 Heartbeats

Comment lines (`: heartbeat\n\n`) every 15-30s prevent intermediaries (proxies, load balancers) from killing idle connections. Standard practice.

§3. Herald-specific applicability analysis

§3.1 Where SSE fits

- **cherald compliance posture stream** — `/v1/sse/compliance` emits posture-change events.
- **sherald safety event stream** — `/v1/sse/safety` emits safety events.
- **bherald budget stream** — `/v1/sse/budget`.
- **rherald risk stream** — `/v1/sse/risk`.
- **iherald incident timeline stream** — `/v1/sse/incidents`.
- **MCP-over-Streamable-HTTP fallback** — Streamable HTTP USES SSE for streaming responses; same plumbing.

§3.2 SSE vs WS vs Streamable HTTP (MCP)

Need	Pick
Browser dashboard, read-only SSE	
Browser chat, interactive	WebSocket
MCP integration	Streamable HTTP (which is SSE under the hood)
High-rate ingest	gRPC client-streaming

§3.3 Multi-tenant + auth

JWT in Authorization: Bearer is preferred but EventSource browser API CAN'T set custom headers. Workarounds:

1. **Token in query** — `?token=<jwt>`. Works everywhere but leaks token to access logs. Mitigation: rotate frequently.
2. **Cookie** — Authorization cookie. Works for browser flows; combine with CSRF protection (Origin check).
3. **Fetch-streams** (modern browsers via Streams API) — can set custom headers but is more complex.

Recommend BOTH cookie + query (cookie for browsers, query for `curl -N`).

§3.4 Idempotency

Read-only; idempotency-by-construction.

§3.5 Resumability

Critical. Each event has `id`. Server's `events_processed` table provides the auth order. On Last-Event-ID: 42, server replays events with `id > 42` (up to a tunable window — e.g. last 1 hour).

§3.6 OTel propagation

Inbound: `traceparent` HTTP header on the initial GET. Outbound: each event's data carries `traceparent` field.

§3.7 Failure modes

1. **Connection pile-up**. Long-lived connections. Limit per-tenant to e.g. 10 concurrent SSE connections.
2. **Proxy timeouts**. Many proxies kill idle TCP at 60-300s. Heartbeats prevent this.
3. **Buffering breaks SSE**. Without `Cache-Control: no-cache` + explicit flush, intermediaries may buffer events. Test against `nginx` + `cloudfront`.

§4. Step-by-step implementation guide for Herald

§4.1 No new dep

Pure Go stdlib. `net/http.ResponseWriter`, `http.Flusher`.

§4.2 New module `commons_sse/`

```
commons_sse/
├── go.mod
├── server/
│   ├── writer.go      # SSE writer; handles framing + heartbeat
│   ├── auth.go        # JWT-from-query OR cookie
│   ├── resume.go      # Last-Event-ID resumption from events_pro
│   └── otel.go
```

```
└─ server_test.go
└─ e2e_real_curl_test.go
```

§4.3 Per-flavor routes

Flavor	Route	Event types
pherald	/v1/sse/events	event.ingested
cherald	/v1/sse/compliance	posture.changed, attested
sherald	/v1/sse/safety	safety.event, oom.killed
bherald	/v1/sse/budget	budget.overrun
rherald	/v1/sse/risk	risk.flagged
iherald	/v1/sse/incidents	incident.created, incident.resolved
scherald	/v1/sse/schedule	job.scheduled, job.completed

§4.4 Event ID convention

Use `events_processed.id` (UUIDv7, sortable). On reconnect with `Last-Event-ID: <uuid>`, server queries `events_processed WHERE id > <uuid> AND tenant_id = <ctx-tenant>` ordered by id.

§4.5 New e2e_bluff_hunt invariants

- **E85** — curl streams: `curl -N 'http://localhost:7104/v1/sse/events?token=<jwt>'` → receives at least one event within 5s.
- **E86** — heartbeat: idle 30s → at least one comment line received.
- **E87** — resumption: client disconnects after `id=42`; reconnects with `Last-Event-ID`; receives events `id > 42`.
- **E88** — auth: query-less + cookie-less → 401.
- **E89** — cross-tenant: Tenant A's token → no Tenant B events.

§4.6 HRD scaffolding

- **HRD-303** — bootstrap `commons_sse/`.
- **HRD-304** — writer + heartbeat.
- **HRD-305** — auth (cookie + query).
- **HRD-306** — resumption.
- **HRD-307** — per-flavor route registration (×7).
- **HRD-308** — OTel + tenant isolation.
- **HRD-309** — e2e_bluff_hunt E85–E89 + mutation gates + spec amendments.
- **HRD-310** — close Wave 4e (SSE slice).

§5. §107 anti-bluff testing strategy

§5.1 Happy-path tests

- **Test 1: curl -N event stream.** Trigger an event via REST → assert curl receives it within 1s.
- **Test 2: heartbeat.** Open stream; wait 30s with no events; assert `≥1 : heartbeat` line.
- **Test 3: resumption.** Open stream; receive events 1..5; disconnect; reconnect with `Last-Event-ID: <id-of-3>`; assert receive 4, 5, plus any newly-arrived.

§5.2 Edge cases

- **Test 4: multi-line data.** Event with `\n` in body → framed as multiple `data: lines`; client reassembles.
- **Test 5: huge event.** 1 MiB event → received complete (no truncation).
- **Test 6: client disconnect mid-stream.** Server detects via `<-ctx.Done()`; cleanup; no goroutine leak.

§5.3 Failure modes

- **Test 7: no auth.** 401.
- **Test 8: cross-tenant.** 0 events received from other tenant.
- **Test 9: behind nginx.** Test with nginx reverse proxy with default config → ensure events still flush.

§5.4 Performance

- **Test 10: 1000 concurrent SSE.** All receive same broadcast event; no missed deliveries.

§5.5 Mutation gates

- Mutation: forget `Flusher.Flush()` → Test 1 buffers; events delayed; FAIL.
- Mutation: skip heartbeat ticker → Test 2 FAIL after proxy timeout.
- Mutation: ignore Last-Event-ID → Test 3 sees duplicates from event 1.

§5.6 Browser-real test

`scripts/e2e_sse_real_browser.sh` — Playwright loads a test HTML page with new `EventSource('/v1/sse/events?token=...')`; asserts `onmessage` fires.

§5.7 Wire-level

- `tcpdump` → assert `Content-Type: text/event-stream + chunked transfer + \n\n` event separators.

§6. Open questions for operator

1. **Heartbeat cadence?** Recommend 30s.
2. **Max connection lifetime?** Recommend 1 hour; force reconnect via `retry` field.
3. **Resumption window?** Recommend last 1 hour of `events_processed` (configurable).
4. **Cookie vs query for auth?** Recommend BOTH supported.
5. **HTTP/1.1 vs HTTP/2 in proxy?** Document recommendation: HTTP/2 (multiplexed; lifts the 6-connections-per-origin limit).

§7. References

(All fetched 2026-05-22.)

- **WHATWG HTML spec.** <https://html.spec.whatwg.org/multipage/server-sent-events.html>
- **MDN EventSource.** https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events

- **MDN Using SSE guide.** https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events
- **Wikipedia.** https://en.wikipedia.org/wiki/Server-sent_events
- **Go stdlib http.Flusher.** <https://pkg.go.dev/net/http#Flusher>
- **2026 Go SSE guide.** <https://oneuptime.com/blog/post/2026-01-25-server-sent-events-streaming-go/view>

§8. Catalogue-check verdict

Per §11.4.74:

- **vasic-digital:** no module.
- **HelixDevelopment:** no module.

Verdict: no external dep needed; implement Herald-internal using Go stdlib net/http. No submodule.